

(https:// /  Q...



HackMD

(https://hackmd.io?utm_source=view-page&utm_medium=logo-nav)

Quiz1 of Computer Architecture (2023 Fall)

Solutions

Problem A

Consider a C implementation of the count leading zero (https://en.wikipedia.org/wiki/Find_first_set#CLZ) function for 64-bit integers. A leading zero is defined as any '0' digit that appears before the first non-zero digit in the binary representation of a number.

Examples:

- Input : $N = 16 \rightarrow$ Output : 59

[illegible]

- Input : $N = 33 \rightarrow$ Output : 58

[illegible]

The implementation utilizes population count (https://en.wikichip.org/wiki/population_count), which counts the number of set bits in the given value.

The source code is listed below, and you shall fill in parts A01 and A02.

```
#include <stdint.h>

uint16_t count_leading_zeros(uint64_t x)
{
    x |= (x >> 1);
    x |= (x >> 2);
    x |= (x >> 4);
    x |= (x >> 8);
    x |= (x >> 16);
    x |= (x >> 32);

    /* count ones (population count) */
    x -= ((x >> 1) & A01 /* Fill this! */ );
    x = ((x >> 2) & 0x3333333333333333) + (x & A02 /* Fill this! */);
    x = ((x >> 4) + x) & 0x0f0f0f0f0f0f0f0f;
    x += (x >> 8);
    x += (x >> 16);
    x += (x >> 32);

    return (64 - (x & 0x7f));
}
```

Obviously, the above C code listing was incomplete, you shall provide the functioned implementations. A01 and A02 are hexadecimal literals (<https://www.includehelp.com/c/hexadecimal-literals.aspx>) in C. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- No space character

You can read the content of Population count (https://www.chessprogramming.org/Population_Count) and find the section titled "The PopCount routine," then select the constants.

```
A01 = ?  
0x5555555555555555  
A02 = ?  
0x3333333333333333
```

Problem B

Consider a C program which converts single precision floating point values to the corresponding bfloat16 (https://en.wikipedia.org/wiki/Bfloat16_floating-point_format), floating-point format.

Single-precision (FP32)	bfloat16 as HEX literals
1.200000	0x3f99999a
1.203125	0x3f9a0000
2.310000	0x4013d70a
2.312500	0x40140000
3.460000	0x405d70a4
3.453125	0x405d0000
5.630000	0x40b428f6

The source code is listed below, and you shall fill in parts B01 and B02.

```

float fp32_to_bf16(float x)
{
    float y = x;
    int *p = (int *) &y;
    unsigned int exp = *p & 0x7F800000;
    unsigned int man = *p & 0x007FFFFF;
    if (exp == 0 && man == 0) /* zero */
        return x;
    if (exp == B01 /* Fill this! */) /* infinity or NaN */
        return x;

    /* Normalized number */
    /* round to nearest */
    float r = x;
    int *pr = (int *) &r;
    *pr &= 0xFF800000; /* r has the same exp as x */
    r /= B02 /* Fill this! */;
    y = x + r;

    *p &= 0xFFFF0000;

    return y;
}

```

Obviously, the above C code listing was incomplete, you shall provide the functioned implementations. B01 and B02 are hexadecimal literals

(<https://www.includehelp.com/c/hexadecimal-literals.aspx>) in C. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- No space character

Reference: Converting float to int in C (<https://onestepcode.com/float-to-int-c/>).

- B01 = ?
0x7F800000
- B02 = ?
0x100

Problem C

Assuming that special values such as NaN and INF do not appear during calculations, the following C code attempts to implement single-precision floating-point multiplication in a minimal way. There is also no overflow.

```

#include <stdio.h>

#include <stdint.h>
uint64_t mask_lowest_zero(uint64_t x)
{
    uint64_t mask = x;
    mask &= (mask << 1) | 0x1;
    mask &= (mask << 2) | 0x3;
    mask &= (mask << 4) | 0xF;
    mask &= (mask << 8) | 0xFF;
    mask &= (mask << 16) | 0xFFFF;
    mask &= (mask << 32) | 0xFFFFFFFF;
    return mask;
}

int64_t inc(int64_t x)
{
    if (~x == 0)
        return 0;
    /* TODO: Carry flag */
    int64_t mask = mask_lowest_zero(x);
    int64_t z1 = mask ^ ((mask << 1) | 1);
    return (x & ~mask) | z1;
}

static inline int64_t getbit(int64_t value, int n)
{
    return (value >> n) & 1;
}

/* int32 multiply */
int64_t imul32(int32_t a, int32_t b)
{
    int64_t r = 0, a64 = (int64_t) a, b64 = (int64_t) b;
    for (int i = 0; i < 32; i++) {
        if (getbit(b64, i))
            r += a64 << i;
    }
    return r;
}

/* float32 multiply */
float fmul32(float a, float b)
{
    /* TODO: Special values like NaN and INF */
    int32_t ia = *(int32_t *) &a, ib = *(int32_t *) &b;

    /* sign */
    int sa = ia >> 31;
    int sb = ib >> 31;

    /* mantissa */
    int32_t ma = (ia & 0x7FFFFFFF) | 0x800000;
    int32_t mb = (ib & 0x7FFFFFFF) | 0x800000;

    /* exponent */
    int32_t ea = ((ia >> 23) & 0xFF);
    int32_t eb = ((ib >> 23) & 0xFF);

    /* 'r' = result */

```

```

int64_t mrtmp = imul32(ma, mb) >> 23;
int mshift = getbit(mrtmp, C01);

int64_t mr = mrtmp >> mshift;
int32_t ertmp = ea + eb - C02;
int32_t er = mshift ? inc(ertmp) : ertmp;
/* TODO: Overflow ^ */
int sr = sa ^ sb;
int32_t r = (sr << C03) | ((er & 0xFF) << C04) | (mr & 0x7FFFFFFF);
return *(float *) &r;
}

```

Obviously, the above C code listing was incomplete, you shall provide the functioned implementations. C01, C02, C03, and C04 are decimal integer literals (https://en.cppreference.com/w/cpp/language/integer_literal) in C. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- No space character

Reference: Floating-Point Numbers (<https://en.algorithmica.org/hpc/arithmetic/float/>).

- C01 = ?
24
- C02 = ?
127
- C03 = ?
31
- C04 = ?
23

Problem D

Let us endeavor to ascertain endianness (<https://en.wikipedia.org/wiki/Endianness>) at compile time. When the need arises for compile-time endianness determination, it typically falls into one of two distinct use cases:

- In select, infrequent scenarios, one must discern endianness to properly structure data layouts, such as when dealing with a union like RGB (https://en.wikipedia.org/wiki/RGB_color_spaces), or when endianness considerations are integral to the code logic.
- In the majority of cases, the primary requirement is to facilitate the conversion between different endiannesses, such as when a communication protocol dictates that a value must be stored in a specific endianness.

In most prevalent use cases, the objective is to facilitate the conversion between little-endian and big-endian formats, as well as potentially to and from the host endianness. For this purpose, we shall introduce endian conversion functions, which shall be denoted by the `end_` prefix.

```
/* Return a value in which the order of the bytes in 4-byte arguments is reversed. */
static inline uint32_t end_bswap32(uint32_t x)
{
    return (x >> 24) | (x >> 8 & D01) | (x << 8 & D02) |
           (x << 24);
}

/* Host to Big Endian 32-bit */
static inline uint32_t end_htobe32(uint32_t n)
{
    union {
        int i;
        char c;
    } u = {1};
    return u.c ? D03 : D04;
}

/* Host to Little Endian 32-bit */
static inline uint32_t end_htole32(uint32_t n)
{
    union {
        int i;
        char c;
    } u = {1};
    return u.c ? D05 : D06;
}
```

You shall provide the functioned implementations. Both `D01` and `D02` are hexadecimal integer literals, meaning that they should start with `0x`. `D03`, `D04`, `D05`, and `D06` are C expressions. You might consider to call `end_bswap32` function when it is necessary. You must obey the following rules when filling them:

- Write shorter code as possible as you can.
- Do necessary casting to eliminate compilation warnings.
- Follow the consistent coding style. That is, we prefer `end_bswap32(n)` to `end_bswap32( n )`. Be aware of the **spaces**! Details determine success or failure.

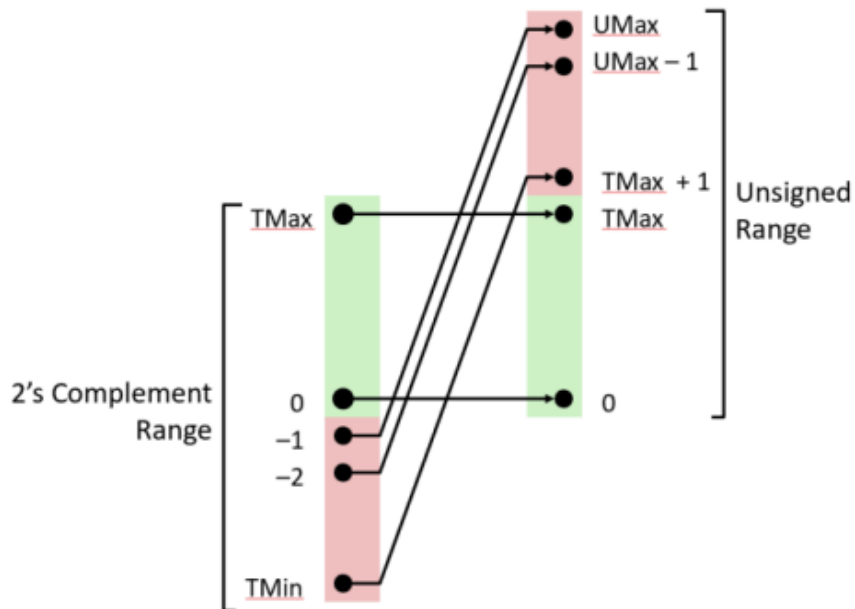
- D01 = ?
0xff00
- D02 = ?
0xff0000
- D03 = ?
end_bswap32(n)
- D04 = ?
n
- D05 = ?
n
- D06 = ?
end_bswap32(n)

Problem E

Arithmetic overflow manifests when the outcome of a computation cannot be adequately represented within the constraints of the prevailing encoding scheme, consequently falling beyond the range of representable values, thus yielding an erroneous result.

Distinct categories of overflow can be delineated as follows:

- **Unsigned Overflow:** Occurs when the resultant value extends beyond the interval $[UMin, UMax]$. An observable indication of unsigned overflow emerges when the addition of two numbers yields a result smaller than either of the operands.
- **Signed Overflow:** Takes place when the outcome extends outside the span of $[TMin, TMax]$. A discernible indicator of signed overflow arises when two numbers of the same sign are added, yet the resultant sum bears the opposite sign.



1. Find the largest 8-bit unsigned numeral c (answer in HEX) such that $c + 0x80$ causes NEITHER signed nor unsigned overflow in 8 bits.

- E01 = ?

0x7F

Unsigned overflow will occur for $c > 0x80$. Signed overflow can only happen if c is negative (also $> 0x80$). Largest is therefore, 0x7F

2. Find the smallest 8-bit numeral c (answer in HEX) such that $c + 0x71$ causes signed overflow, but NOT unsigned overflow in 8 bits.

- E02 = ?

0xF

For signed overflow, need $(+) + (+) = (-)$. For no unsigned overflow, need no carryout from MSB. The first $(-)$ encoding we can reach from $0x71$ is $0x80$.

$0x80 - 0x71 = 0xF$.

Problem F

The subsequent function, denoted as $absf$, yields the absolute value of a single-precision floating-point number. What is the hexadecimal literal representation of the value denoted as F01?

```
#include <stdint.h>
float absf(float x)
{
    uint32_t mask = F01;
    union {
        uint32_t i;
        float f;
    } u = {.f = x};
    u.i &= mask;
    return u.f;
}
```

- F01 = ?

0x7fffffff